

PLP-Lab-Aula 11 - Material Complementar

Paradigma Funcional - Linguagem Haskell

1. Objetivos desta Aula

- Desafio de aprender uma linguagem de programação nova
- Conhecer uma linguagem de programação do Paradigma Funcional

2. Roteiro da Aula

- **PRIMEIRO**: realize os exercícios abaixo e já procure ir entendendo a linguagem Scala. NÃO VÁ DIREITO para a Lista!!!!
- **SEGUNDO**: procure o máximo de materiais da **Linguagem Haskell** e **ESTUDE** estes materiais junto com seu grupo.
- **TERCEIRO**: Tente fazer a Lista 11. Se tiver dúvidas, o(a) professor(a) poderá “dar alguma dica” para ajudar, mas no primeiro momento não dará a solução. Isso para que você e seu grupo tenham a experiência de aprender uma linguagem nova “por conta” (algo que ocorrerá por muitas vezes quando você terminar a faculdade!!!).

(Obviamente, o(a) professor(a) poderá tirar suas dúvidas em um momento posterior).

3. Ambiente para Executar Haskell

https://www.onlinegdb.com/online_haskell_compiler

4. Material para Linguagem Haskell

<https://www.inf.ufpr.br/andrey/ci062/ProgramacaoHaskell.pdf>

<http://www.decom.ufop.br/romildo/2018-1/bcc222/slides/progfunc.pdf>

Para aprender mais sobre Haskell, acesse:

1. **Learn You a Haskell for Great Good!:** Este é um livro online gratuito que é frequentemente recomendado para iniciantes em Haskell. Ele aborda os conceitos básicos de Haskell de uma maneira acessível e divertida.

1. [Learn You a Haskell for Great Good!](https://learnyouahaskell.com/)

<https://learnyouahaskell.com/>

2. **Real World Haskell:** Este livro é uma ótima opção para quem quer aprender Haskell de maneira prática, aplicando os conceitos a projetos do mundo real.

1. Real World Haskell

<https://book.realworldhaskell.org/read/>

3. **Haskell Programming from First Principles:** Este é outro livro popular que aborda Haskell desde o início, sem pressupor qualquer conhecimento prévio de programação funcional.

1. [Haskell Programming from First Principles](https://haskellbook.com/)

<https://haskellbook.com/>

4. **Haskell Wiki:** O Haskell Wiki é uma excelente fonte de informações sobre a linguagem Haskell. Ele inclui tutoriais, exemplos de código e outras informações úteis.

1. Haskell Wiki

2. <https://wiki.haskell.org/Haskell>

5. **Haskell.org:** O site oficial da linguagem Haskell tem uma série de recursos, incluindo documentação, tutoriais e links para bibliotecas.

1. [Haskell.org](https://haskell.org)

[Haskell Language](https://haskell.org)

5. Exemplos em Haskell

Exemplo 1 – Fatorial de um número digitado pelo usuário

```
fatorial :: Integer -> Integer
```

```
fatorial 0 = 1
```

```
fatorial n = n * fatorial (n - 1)
```

```

main :: IO ()
main = do
    putStrLn "Digite um número para calcular o fatorial:"
    input <- getLine
    let numero = read input :: Integer
    putStrLn $ "O fatorial de " ++ show numero ++ " é " ++ show
(fatorial numero)

```

Exemplo 2 – Cálculo da soma de dois vetores de tamanho 6

```

somaVetores :: [Int] -> [Int] -> [Int]
somaVetores [] [] = []
somaVetores (x:xs) (y:ys) = (x + y) : somaVetores xs ys

main :: IO ()
main = do
    putStrLn "Digite os elementos do primeiro vetor separados por
espaço:"
    input1 <- getLine
    let vetor1 = map read $ words input1 :: [Int]

    putStrLn "Digite os elementos do segundo vetor separados por
espaço:"
    input2 <- getLine
    let vetor2 = map read $ words input2 :: [Int]

    if length vetor1 /= 6 || length vetor2 /= 6
    then putStrLn "Os vetores devem ter tamanho 6!"
    else do
        let resultado = somaVetores vetor1 vetor2
        putStrLn $ "A soma dos vetores é: " ++ show resultado

```

Exemplo 3 – Multiplicação de duas matrizes de ordem 3

Este programa define funções para calcular o produto escalar de duas listas, calcular a transposta de uma matriz e multiplicar duas matrizes. Em seguida, na função principal `main`, duas matrizes de ordem 3 são definidas e multiplicadas. Os resultados são exibidos no console.

```
-- Função para calcular o produto escalar de duas listas
produtoEscalar :: Num a => [a] -> [a] -> a
produtoEscalar xs ys = sum $ zipWith (*) xs ys

-- Função para calcular a transposta de uma matriz
transposta :: [[a]] -> [[a]]
transposta ([]:_) = []
transposta x = (map head x) : transposta (map tail x)

-- Função para multiplicar duas matrizes
multiplicaMatrizes :: Num a => [[a]] -> [[a]] -> [[a]]
multiplicaMatrizes a b = [[produtoEscalar linha coluna | coluna <-
transposta b] | linha <- a]

main :: IO ()
main = do
    let matriz1 = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
    let matriz2 = [[9, 8, 7], [6, 5, 4], [3, 2, 1]]

    putStrLn "Matriz 1:"
    mapM_ print matriz1
    putStrLn "Matriz 2:"
    mapM_ print matriz2

    let resultado = multiplicaMatrizes matriz1 matriz2

    putStrLn "Resultado da multiplicação das matrizes:"
    mapM_ print resultado
```

Exemplo 4 – Calcular a posição da sub string “fofo” na string

"Gatossaoosanimaismaisfofosdaterra".

```
-- Função auxiliar para verificar se uma substring está presente
em uma determinada posição da string

verificarSubString :: String -> String -> Int -> Bool

verificarSubString [] _ _ = True
verificarSubString _ [] _ = False
verificarSubString (x:xs) (y:ys) pos
    | x == y = verificarSubString xs ys (pos + 1)
    | otherwise = False

-- Função principal para encontrar a posição da substring "fofo"
na string

posicaoSubString :: String -> Int -> Maybe Int

posicaoSubString [] _ = Nothing
posicaoSubString str pos
    | verificarSubString "fofo" str pos = Just pos
    | otherwise = posicaoSubString (tail str) (pos + 1)

main :: IO ()
main = do
    let stringOriginal = "Gatossaoosanimaismaisfofosdaterra"
    let posicao = posicaoSubString stringOriginal 0
    case posicao of
        Just p -> putStrLn $ "A substring 'fofo' está na posição
" ++ show p ++ " da string."
        Nothing -> putStrLn "A substring 'fofo' não foi encontrada
na string."
```

DESAFIO: Faça um programa em Haskell que calcule a solução para o problema da Torre de Hanói com 3 discos.